Ders 5 Çalışma Özeti

Ders 5 Çalışma Özeti

Genel Konular

- Iterator kalıbı (davranışsal)
  - Amaç (ekrandan, 00:10:00): parçalardan oluşan bir bileşenin parçalarını, bileşenlerin iç yapısını açığa çıkarmadan, istenilen bir biçimde dolaşmak. En basit örnek bir veri yapısı içinde dolaşmaktır.
  - Sorun: veri yapılarının çok sayıda çeşidi vardır. Listeler tek yönlü, çift yönlü, dairesel (circular) ya da değil olabilir; bu, yalnız listede dört varyasyon eder. Ağaçlar ikili (binary) ya da n-li (n-ary) olabilir; çocuktan ebeveyne, yapraktan köke işaretçi tutan türleri vardır. Tablo ve map yapıları da vardır.
  - Aynı yapıda bile farklı dolaşım algoritmaları vardır. İkili ağaçta LDR (Left-Data-Right), RDL, DLR ve DRL sıraları ayrı algoritmalardır. Veri yapısının arayüzü bu farklı dolaşım türleriyle şişirilmek istenmez (ekrandan, 00:10:00).
  - Java'da her Collection'dan kendini gezmek üzere bir `Iterator` istenebilir; ancak bu nesne Iterator tasarım kalıbına tam uymaz, daha basittir. Basit demek kötü demek değildir. KISS (Keep It Simple, Stupid): Java'nın Iterator'u işinize yarıyorsa onu kullanın. Yaramıyorsa ya da dilde böyle bir altyapı yoksa kalıp kurulur.
  - 1. çözüm (ekrandan, 00:15:00): `List {Count(), Append(Element), Remove(Element), ...}` ve ona `list` ilişkisiyle bağlı `ListIterator {First(), Next(), IsDone(), CurrentItem(); index}`. Güçlü yönü: veri yapısı ile dolaşım biçimi birbirinden soyutlanır, uyum (cohesion) artar. Zayıf yönü: belli bir veri yapısı ile belli bir dolaşım biçimi arasında bağlılık oluşur.
  - Bu bağlılığı azaltmak için genel bir liste arayüzü ve genel bir dolaşım arayüzü oluşturulur. Hocanın ilk aklına Abstract Factory geldi, ama veri yapısı ailelerinin üyeleri eş biçimli olmadığı için vazgeçti; Factory Method uygundur.
  - 2. çözüm (ekrandan, 00:20:00): `Client`, soyut `AbstractList {CreateIterator(), Count(), Append(Item), Remove(Item), ...}` ve soyut `Iterator {First(), Next(), IsDone(), CurrentItem()}` ile konuşur. Alt sınıflar `List`/`SkipList` ve `ListIterator`/`SkipListIterator` çiftler hâlinde birbirine bağlıdır. `CreateIterator()` bir sanal fabrika metodudur (Factory Method).
  - Kalıp yapısı, bu çözümün genelleştirmesidir: soyut bir bütün (aggregate) sınıfı `CreateIterator()` sunar, soyut iterator sınıfı dolaşmanın temel yeteneklerini tanımlar, somut alt sınıflar ayrıntıyı kodlar. İstemci yalnız soyut sınıflarla konuştuğu için yeni alt sınıflar onu etkilemez.
- Iterator gerçekleme kararları
  - Dolaşım sırasında parçaların durumunun değiştirilmesine izin verilecek mi? İzin verilmeyecekse iterator yapının kendisi yerine klonu üzerinde dolaşabilir.
  - Dolaşma algoritmasını kim gerçekleyecek: veri yapısı mı, iterator mı? (Örneğin ağaçtaki LDR, DLR seçimi nerede kodlanacak?)
  - Elemanlar istekçiye birer birer mi (next, next, …) verilecek, yoksa hepsi birden (örneğin bir dizi içinde) mi? Hepsini birden vermek daha uzun sürer ama istekçiye serbestlik verir. Birer birer almak başarımı iyileştirebilir; elemanlar ağ üzerinden geliyorsa ve haberleşme masraflıysa arka planda önceden çekme (prefetch) mekanizması kurmak yararlı olabilir.
  - Bu soruların tek doğru yanıtı yoktur; problem ya da uygulama alanı neyi gerektiriyorsa o seçilir.
- Iterator örneği: birleşen iki lokantanın menüsü (Head First Design Patterns'tan)

  - Problem: çevrim içi yemek siparişi işine hızlı girmek için biri kahvaltıda (Pancake House), biri yemekte (Diner) uzman iki firma satın alınır. `MenuItem` sınıfı ortaktır, ama iki firma menüyü farklı gerçeklemiştir. Ayrıca vejetaryen seçenekler işaretlenmiştir ve "yalnız vejetaryen seçenekleri göster" isteği vardır.
  - `MenuItem` (ekrandan, 00:30:00): `name`, `description`, `vegetarian`, `price` alanları, getter'lar ve `toString()`: `String info = name + "\t" + description + "\t" + price; if (vegetarian) info += "\t" + "(Vegetarian)"; return info;`.
  - `DinerMenu` menüyü 6 elemanlık bir `MenuItem[]` dizisinde tutar; `addItem` dizi doluysa eklemez. `getMenuItems()` diziyi olduğu gibi döndürür. `PancakeHouseMenu` ise `ArrayList<MenuItem>` kullanır ve `getMenuItems()` listeyi döndürür.
  - İlk ("quick and dirty") çözüm, `WaitressQuickAndDirty` (ekrandan, 00:36:30): her menü için ayrı döngü yazılır; `ArrayList` için `breakfastItems.size()` ve `get(i)`, dizi için `dinerItems.length` ve `dinerItems[i]`. Vejetaryen menü aynı döngülerin içine `if (item.isVegetarian())` eklenmiş kopyasıdır; `printBreakfastMenu()` ve `printDinerMenu()` da aynı kodun parçalarını tekrarlar.
  - Sakıncaları: iki yapı iki ayrı biçimde ele alınır; aynı kod birçok yerde tekrarlanır (duplicate code kusuru); her yeni firma için yeni kod gerekir; bir hata ya da değişiklik birkaç metotta birden düzeltilmelidir.
  - Iterator ile çözüm (ekrandan, 00:41:00): `«interface» Menu {createIterator() : Iterator<MenuItem>}`; `DinerMenu` ve `PancakeHouseMenu` bunu gerçekler. `«interface» Iterator<MenuItem> {hasNext() : boolean, next() : MenuItem}` (Java'nın `java.util.Iterator`'u); `DinerMenuIterator` ve `PancakeHouseMenuIterator` bunu gerçekler. `Waitress` yalnız `Menu` ve `Iterator` arayüzlerini kullanır.
  - `Menu` arayüzü (ekrandan, 00:40:00): `public java.util.Iterator<MenuItem> createIterator();`. Üye alan ve somut metot gerekmediği için soyut sınıf değil arayüz seçildi.
  - `DinerMenuIterator` (ekrandan, 00:44:20): kurucu `MenuItem[] list` alır; `next()`: `MenuItem menuItem = list[position]; position = position + 1; return menuItem;`; `hasNext()`: `if (position >= list.length || list[position] == null) return false; else return true;`. `PancakeHouseMenuIterator` aynı işi `ArrayList`'in `get(position)` ve `size()` metotlarıyla yapar.
  - Ders sonrası not: Hoca `hasNext()` için "sona ulaşmamışsa false, ulaşmışsa true döndürüyor" dedi (00:44:28, 01:45:42); ekrandaki kodda (00:44:20) doğrusu tersidir: sona ulaşılmışsa ya da eleman `null` ise `false`, aksi hâlde `true`.
  - `Waitress` (ekrandan, 00:50:00): `printMenu(Iterator<MenuItem> iterator)` `while (iterator.hasNext())` döngüsünde her elemanı `printMenuItem`'a verir; `printMenuItem` adı, fiyatı ve tanımı yazar. `printVegetarianMenu()` önce kahvaltı evinin, sonra yemek evinin `createIterator()` sonucunu `printVegetarianMenu(Iterator)`'a verir; bu metot yalnız vejetaryen elemanları yazar.
  - Kalan sakınca: `printMenu` ile `printVegetarianMenu` arasındaki tek fark bir `if` komutudur. Iterator'a "bütün üyelerde / vejetaryen üyelerde dolaş" seçeneği eklenirse çözülür, ama hazır `java.util.Iterator`'a metot eklenemez; kendi iterator sınıfını yazmak gerekir.
- Memento kalıbı (davranışsal)
  - Ad: sözlük anlamı "anı parçası". Hoca adı, yakın geçmişini hatırlayamayan ve küçük notlarla hareket eden bir kahramanı anlatan "Memento" (Türkçede "Akıl Defteri") filmiyle tanıttı.
  - Amaç: bir nesnenin durum bilgisini, sarmalama (encapsulation) ilkesini bozmadan elde etmek ve saklamak; böylece gerekirse önceki durumlarından birine dönülebilir. Saklanan durum bilgisi dışarıdan değiştirilememeli; sahte (forge edilmiş) durum bilgisi verilememelidir. Durumun tümü ya da yalnız gerekli kısmı saklanabilir.
  - Kullanım alanları: diske saklamak, ağ üzerinden aktarmak, geçirilen aşamaların günlüğünü tutmak, işlem (transaction) mantığında hata olunca geri alma (rollback), düzenleme işlemlerinde geri alma (undo), yedekten geri yükleme.
  - Kalıp yapısı (ekrandan, 01:20:00): `Originator {SetMemento(Memento m), CreateMemento(); state}`; `CreateMemento` notu `return new Memento(state)`, `SetMemento` notu `state = m->GetState()`. `Memento {GetState(), SetState(); state}`. `Caretaker`, `memento` ilişkisiyle Memento'ları tutar (bire-çok).
  - Bileşenler (ekrandan, 01:20:00): Originator durum bilgisi saklanacak olan nesnedir; Memento durum bilgisinin gerekli kısımlarını saklar; Caretaker gerekli anlarda du-

rum bilgisinin bir kopyasını alır ve elindeki kopyalardan birini geçerli durum hâline getirir. Bunu Memento'yu değiştirerek değil, Memento'yu Originator'ın `SetMemento` metoduna vererek yapar. `GetState`/`SetState`'i yalnız Originator kullanmalıdır.
  - Sıralama şeması (ekrandan, 01:22:00): `aCaretaker` → `anOriginator.CreateMemento();` Originator `new Memento` ile `aMemento`'yu oluşturur ve `SetState()` çağırır. Bir süre sonra `aCaretaker` → `anOriginator.SetMemento(aMemento);` Originator `aMemento.GetState()` çağırır.
  - Ne zaman: durumun tümü ya da bir kısmı sonra geri dönmek için saklanacaksa ve duruma doğrudan bir işaretçi ya da public erişim veri gizliliğini bozacaksa. Veri gizliliği gerekmiyorsa ek kalıp kurmaya gerek yoktur.
- Memento'da veri gizliliğinin gerçeklenmesi
  - Zorluk: bir metot public ise herkes, private ise kimse çağıramaz; "yalnız şu sınıf çağırabilsin" diyen seçici bir erişim mekanizması dillerin çoğunda yoktur.
  -
  - C++: `friend` yapısı. Memento'nun metotları private yapılır ve Originator Memento'nun arkadaşı (friend) ilan edilir; arkadaş sınıf private metotlara erişebilir.
  - Ders sonrası not: Hoca sözdizimini "public class memento, friend originator" diye kabaca söyledi; C++'ta bu, Memento sınıfının içinde `friend class Originator;` bildirimiyle yapılır.
  - Java seçenek 1, paket (package) erişimi: erişim belirteci yazılmayan üyeler başka paketteki sınıflara private gibi davranır. Ama jar dosyasındaki paketler `sealed` işaretlenmezse dışarıdan biri pakete sınıf ekleyip bu üyelere erişebilir.
  - Java seçenek 2, iç sınıf: Memento, Originator'ın iç sınıfı yapılır. Hocanın deneyimi: durum (`State`) sınıfı `Cloneable` gerçeklemeli ve Memento'ya durumun kopyası (klonu) verilmelidir; yoksa geri dönüş çalışmaz.
  - Kod (dp.mementoV2.innerClass): `State implements Cloneable` (ekrandan, 01:30:00) `private String durum`, public `getDurum()`, paket erişimli `setDurum(String)` ve yeni bir `State` oluşturup durumunu kopyalayan `public State clone()`. `Originator` (ekrandan, 01:32:50) `modify(String str)` ile `state.setDurum(str)` yapar, `toString()` `state.getDurum()` döndürür; iç sınıf `public class Memento { private State state; private State getState() {...} ... }`. `createMemento()` boş bir Memento oluşturup durumunu kendi durumunun kopyasıyla ayarlar; `setMemento(m)` Memento'nun durumunu kendi `state`'ine atar.
  - Başka pakette kullanmak için iç sınıf `import ...Originator.Memento` biçiminde ayrıca import edilmelidir.
- Memento kodunun denenmesi
  - `Caretaker.test()`: `subject.modify("ilk durum")`, `mem = subject.createMemento()`, `subject.modify("yeni durum")`, `subject.setMemento(mem)`, yazdır. Çıktı "ilk durum"dur.
  - `clone()` yerine `return this` yazılırsa çıktı "yeni durum" olur; geri dönüş gerçekleşmez. Klon döndürülünce yine "ilk durum" çıkar.
  - Caretaker, Memento'nun işaretini alabilir ama ondan yalnız `Object` sınıfının metotlarını çağırabilir; Memento'yu doğrudan değiştiremez.
  - `testBroken()` (ekrandan, 01:40:00): ikinci bir `fakeSubject` oluşturulur, `"sahte durum"` verilir, onun Memento'su (`memFake`) asıl `subject`'e `setMemento` ile verilir. Bu deneme başarılı olur: Memento doğrudan değiştirilemese de dolaylı olarak başka bir nesnenin Memento'su verilebilir.
  - Hocanın önerdiği önlem: Originator'a kurucuda rastgele ilklendirilen, getter'ı olmayan private `final` bir kimlik (ID) alanı konur; iç sınıf Memento bu kimliği okuyabilir (iç sınıf dış sınıfın private üyelerine erişir). `setMemento`, Memento'daki kimlik kendi kimliğiyle aynı değilse durumu atamaz. Bu derste kodlanmadı.
- Memento örneği: belge işleme programında geri alma ve yineleme (ekrandan, 01:46:30)
  - `Controller` (1) → (*) `Document`; not: Controller'ın `DocumentMemento` ile hiçbir ilişkisi yoktur. `Document {- currentState : DocumentMemento, - states : List<DocumentMemento>, - position : int; + undo(), + redo(), + anyOperation()}` ve `DocumentMemento {- documentState}`.
  - `undo()`: `position -= 1; currentState = states.get(position);`. `redo()`: `position += 1; currentState = states.get(position);`. `anyOperation()`: `states.add(currentState);`.
  - Memento'ların yönetimini Controller da yapabilirdi; burada Document yapar. Bu ör-

neğin kodu yazılmadı.

- Observer kalıbı (davranışsal)
  - Amaç: bir nesnenin durumunda değişiklik olduğunda onunla ilişkili bütün nesnelerin haberdar edilmesi; böylece onlar gerekli eylemleri alabilir. Haberdar etme ilişkileri kodlama anında değil çalışma anında kurulup bozulabilmelidir.
  - Örnek: bir hesap tablosundaki veriye (50-30-20) bağlı bir pasta grafik ve bir çubuk grafik; veri değişince grafikler güncellenir.
  - Kalıp yapısı (ekrandan, 02:10:00): soyut `Subject {Attach(Observer), Detach(Observer), Notify()}`, `observers` ilişkisiyle soyut `Observer {Update()}`'a bağlı. `Notify` notu: `for all o in observers { o->Update() }`. `ConcreteSubject {GetState(), SetState(); subjectState}`, `GetState` notu `return subjectState`. `ConcreteObserver {Update(); observerState}`, `subject` ilişkisiyle ConcreteSubject'e bağlı; `Update` notu `observerState = subject->GetState()`.
  - Subject ile Observer arasında çoka-çok ilişki vardır. `Attach`, `Detach`, `Notify` somut metot olarak soyut Subject'te kodlanır; Observer ise bir arayüz olabilir.
  - ConcreteObserver'dan ConcreteSubject'e ilişki her zaman gerekmez; durum ayrı bir sınıfta sarmalanmışsa observer o durumla ilişkili olabilir.
  - Hoca, şemadaki `observerState = subject->GetState()` notunu yanıltıcı buldu: observer'ın durumu subject'in durumunun kopyası olmak zorunda değildir; observer değişen bilgiyle kendi işlemlerini yapar.
- Observer gerçekleme ayrıntıları
  - Pull model (ekrandan, 02:20:00): gözlenen yalnız "değiştim" der, gözleyiciler ilgili durumu gözlenenden alır. Artıları: gözlenenin gözleyicilerle bağlantısı azalır (bağlaşım yok olmaz); gözleyici durumun yalnız istediği kısmını alır, bu da ağda daha az haberleşme demektir. Eksisi: gözleyici neyin değiştiğini kendisi çıkarmak zorundadır.
  - Push model: gözlenen, değişiklik hakkında ayrıntılı bilgi gönderir; en basit biçimde kendisini ya da bütün durumunu `Notify`/`Update` ile iletir. Haberleşme masrafı daha fazladır. Durumla birlikte bir değişiklik maskesi (change mask, her alan için bir bit) gönderilirse gözleyici neyin değiştiğini kolayca anlar.
  - İki modelin artıları ve eksileri birbirinin tersidir; problem neyi gerektiriyorsa o seçilir.
  - Aynı türden gözlenenlerin tümünün gözleyicileri ortaksa gözleyiciler statik bir veri yapısında tutulabilir (ekrandan, 02:20:00).
  - Tetikleme: `Notify`'ı kim çağıracak? Subject çağırırsa durumu değiştiren her metodun sonuna `Notify` eklenmelidir; durum çok sık değişiyorsa yoğun haberleşme trafiği olur; mesajları biriktirmek subject'i karmaşıklaştırır. İstemci (client) çağırırsa bunu unutabilir. Gözleyiciler sabit aralıklarla sorgularsa verimsiz olur. Haberleşme masrafında optimizasyon için en uygun yer yine gözlenen nesnedir.
  - Ne zaman: bir soyutlamanın birbirine bağımlı birden fazla parçası varsa; bir nesnedeki değişiklik kodlama anında bilinmeyen sayı ve türde nesneyi etkileyecekse; bir nesne başka nesneleri ek bağlaşım olmadan haberdar edecekse.
- Observer örneği: müşteri ilişkileri programı
  - Problem (ekrandan, 02:32:30): yeni müşteri eklenince "hoş geldiniz" e-postası gönderilecek ve ikamet adresinin doğruluğuna bakılacak. Çözüm 1: `Customer.addCustomer()` `WelcomeLetter` ve `AddrVerification`'ı sabit biçimde çağırır.
  - Gereksinim değişir: müşteri bir mağazanın 10 km yarıçaplı dairesi içindeyse indirim kuponlu mektup gönderilecek. Bu kampanyaların sonu gelmeyeceği için altyapı kurulur. Brick and mortar: hem sanal hem fiziksel ticaret yapan bir kurumun fiziksel binalarda yaptığı ticaret (brick: tuğla, mortar: harç).
  - Neden Command değil: Command'da komut elle verilir; burada istenen, bir durum değişikliğinde (müşteri eklenmesi) sistemin haberdar olup işlemleri otomatik yapmasıdır.
  - Çözüm 2 (ekrandan, 02:40:00): `Customer {- myObs : Vector<ICObserver> {static}; + attach(ICObserver) {static}, + detach(ICObserver) {static}, + getState(), + notifyObs()}`; `«interface» ICObserver {+ update(myCust : Customer)}`; `AddrVerification`, `BrickAndMortar`, `WelcomeLetter` bunu gerçekler. Yalnız müşteri nesneleri izleneceği için ayrı bir soyut Subject üst sınıfı konmadı.
  - Kod (ekrandan, 02:42:15): `private static Vector<ICObserver> myObs = new Vector<ICObserver>();` (gözleyiciler ortak olacak), `attach` → `myObs.addElement(o)`, `detach` → `myObs.remove(o)`, `notifyObs()` → `for (ICObserver obs : myObs) {`

`obs.update(this); }`. `getState()` ve gözleyicilerin `update` gövdeleri iskelet (TODO) olarak bırakıldı.
  - `update` bütün `Customer` nesnesini aldığı için örnek push modelle çalışır.

Hocanın Özellikle Vurguladığı Kısımlar

- KISS: Java'nın hazır Iterator'u işinizi görüyorsa onu kullanın
  - Kalıp ancak hazır altyapı yetmiyorsa ya da dil böyle bir altyapı sunmuyorsa kurulmalıdır.
- Iterator kalıbının "bildik" görünse de önemli gerçekleme ayrıntıları vardır
  - Değişikliğe izin, algoritmanın yeri, elemanların birer birer mi toplu mu verileceği kararları probleme göre verilmelidir.
- En odaklanmış işleri ayrı metotlara almak tavsiye edilir
  - `printMenuItem`'ın ayrı metot olması sayesinde `printVegetarianMenu` aynı yazdırma kodunu tekrar yazmadan kullanabildi.
- Memento'nun amacı veri gizliliğini korumaktır; söylemesi kolay, kodlaması zordur
  - Java'da iç sınıf çözümünde durum klonlanmazsa geri dönüş çalışmaz. Hoca bunu bulmak için çok zaman harcadığını söyledi.
  - İç sınıf çözümü bile başka bir Originator'ın Memento'sunun verilmesini engellemez; kimlik denetimi ek bir önlemdir.
- Observer'da en az pull/push seçimi kadar önemli olan, `Notify`'ın kim tarafından ve nasıl tetikleneceğidir
- Bağlaşım ya vardır ya yoktur
  - Bir metot çağrılsın ya da on metot, ilişki yine vardır; pull model bağlaşımı azaltır ama yok etmez.
- Kalıbı kurmaya zahmet edilecekse özgün hâliyle kurmak daha mantıklıdır
  - Bir subject türünün yalnız belli observer türleri olacaksa basitleştirme yapılabilir, ama bunun hep böyle kalacağının garantisi yoktur.
- Birbirine benzeyen kalıplar ayrıntılarda ayrılır
  - Metotların hangi sıklıkla çağrıldığı ve kalıbın hitap ettiği çekirdek problem farklıdır. Bir problemi birden fazla kalıp çözebilir; önemli olan seçimin gerekçesidir.

Kısa Tekrar Notları

- Iterator: bütünün parçalarını iç yapıyı açmadan dolaşmak; dolaşım sorumluluğu veri yapısından ayrı bir sınıfa alınır.
- 1. çözüm: `List` + `ListIterator`; zayıf yönü somut yapı ile somut iterator arasındaki bağlılık.
- 2. çözüm/kalıp: soyut bütün + `CreateIterator()` (Factory Method) + soyut Iterator (`First`, `Next`, `IsDone`, `CurrentItem`).
- Gerçekleme kararları: değişikliğe izin (gerekirse klon üzerinde dolaş), algoritma nerede, birer birer mi toplu mu.
- Menü örneği: dizi tabanlı `DinerMenu`, `ArrayList` tabanlı `PancakeHouseMenu`; ortak `Menu.createIterator()` ve `java.util.Iterator<MenuItem>` (`hasNext`, `next`).
- Memento: durumu sarmalamayı bozmadan sakla, sonra geri dön; Originator, Memento, Caretaker.
- Memento erişim denetimi: C++ `friend`, Java paket erişimi (sealed jar gerekir) ya da iç sınıf.
- Java iç sınıf çözümünde durum klonlanmalı; `return this` ile geri dönüş çalışmaz.
- Sahte Memento açığı: başka Originator'ın Memento'su verilebilir; önlem private final rastgele kimlik.
- Undo/redo: `states` listesi, `position` sayacı, `currentState`.
- Observer: Subject (`Attach`, `Detach`, `Notify`) ve Observer (`Update`); çoka-çok ilişki.
- Pull: az bağlaşım, seçici alma, ama değişikliği gözleyici bulur. Push: ayrıntılı bilgi, daha çok haberleşme; change mask yardımcı olur.
- Tetikleme: subject (her setter sonunda Notify), istemci (unutulabilir) ya da gözleyici (periyodik, verimsiz).
- Müşteri örneği: `Customer` statik `Vector<ICObserver>`, `notifyObs()` her gözleyiciye `update(this)` der; push model.

Detaylı Açıklamalar

- Iterator kalıbının çıkış noktası, bir bütünü oluşturan bütün parçalar üzerinde aynı işlemi yapma ihtiyacıdır. Bunu yaparken bütünün iç yapısı açığa çıkarılmamalıdır. Veri yapıları çok çeşitlidir: tek yönlü, çift yönlü ve dairesel bağlı listeler; ikili ve n-li ağaçlar; ebeveyn ya da kök işaretçisi tutan ağaçlar; tablo ve map yapıları. Aynı yapının da birden fazla dolaşım algoritması olabilir. Hoca LDR'yi tahtada küçük bir ağaçla gösterdi: kökten başlanır ve gidilebildiği kadar sola gidilir. Solu olmayan düğümün verisi (3) alınır; sağı da yoksa bir üste dönülür. Orada veri (5) alınır, sağ çocuğa gidilir (6), sonra köke dönülüp kökün verisi (9) alınır ve sağ alt ağaca geçilir. Bu geri dönüşler ya ebeveyn işaretçisiyle ya da bir yığına push/pop ile yapılabilir. RDL, DLR ve DRL de ayrı algoritmalardır. Bütün bu algoritmaları veri yapısının arayüzüne eklemek arayüzü şişirir.
- İlk çözümde dolaşım işi `ListIterator` sınıfına alınır. Bu, "neyin değişeceğini bul ve sarmala" ilkesine uyar ve listenin uyumunu artırır: liste yalnız kendi içeriğiyle ilgili işleri yapar. Ama somut liste ile somut iterator arasında yeni bir bağlılık oluşur. İkinci çözüm, istemciyi soyut liste ve soyut iterator ile konuşturur. Her somut liste kendine uygun somut iteratoru `CreateIterator()` fabrika metoduyla üretir. Somut liste ile onun iteratoru arasındaki ilişki yine sürer ve sürmek zorundadır: belli bir yapıda nasıl dolaşılacağının ayrıntıları en iyi o yapıya özel bir iteratorda sarmalanır.
- Menü örneğinin ilk çözümünde `WaitressQuickAndDirty` sınıfı, `PancakeHouseMenu`'nün döndürdüğü `ArrayList`'i ve `DinerMenu`'nün döndürdüğü diziyi ayrı döngülerle dolaşır. Bütün menü, vejetaryen menü, yalnız kahvaltı ve yalnız yemek menüsü metotları aynı kodun parçalarını tekrar eder. Iterator ile çözümde `Menu` arayüzü yalnız `createIterator()` ekler. `PancakeHouseMenu.createIterator()` kendi listesini vererek bir `PancakeHouseMenuIterator`, `DinerMenu.createIterator()` kendi dizisini vererek bir `DinerMenuIterator` oluşturur. Hangi menünün hangi yapıyı kullandığının karmaşıklığı iteratorlara taşınır; ikisi de aynı hizmeti (`hasNext`, `next`) sunar. Hoca `java.util.Iterator`'u gerçeklemenin iki kolaylığına değindi: Java'nın generik desteği (`Iterator<MenuItem>`) ve bu altyapının Java döngüleriyle birlikte kullanılabilmesi. Java'nın iterator kuralına göre `next()` çağrılmadan `remove()` yapılamaz; örnekte `remove` kullanılmadı.
- Vejetaryen sorunu için iterator'a bir yapılandırma ("bütün üyeler / vejetaryen üyeler") eklemek yeterlidir, ama hazır Java iteratoruna metot eklenemez. Hoca, sırf vejetaryen seçeneği için kendi iteratorunu yazmaya değmeyeceğini söyledi. Glutensiz, çocuk menüsü, çölyak hastaları için menü gibi seçenekler artacaksa bir Boolean ya da enum yetmez; enum ile switch-case yapısına gidilir. O durumda daha iyi bir nesneye dayalı tasarım ya da başka bir kalıp gerekir.
- Memento kalıbında Originator `CreateMemento()` ile durumunun bir kopyasını içeren bir Memento üretir; Caretaker bu Memento'ları toplar. Caretaker bunların günlüğünü tutabilir, geri alma işlemleri yapabilir, ama Memento'yu değiştiremez. Geri dönmek istediğinde Memento'yu Originator'ın `SetMemento()` metoduna verir; Originator Memento'dan durumu okuyup kendi setter'larıyla günceller. Hoca slayttaki sıralama şemasında dönüş oklarının çizilmediğini ve `new Memento` mesajının kesikli çizgiyle gösterildiğini belirtti. Bunu, Dörtlü Çete (Gang of Four) kitabının UML gösterimi tam oturmadan yazılmış olmasına bağladı.
- Java iç sınıf çözümünde iç sınıf `Memento` public tanımlanabilir, çünkü metotları private'tır. Dış sınıf iç sınıfın private üyelerine erişebilir, ama başka sınıflar erişemez. Caretaker Memento referansını tutar ve geri verir, ama içini okuyup değiştiremez. Hoca `State`'in klonlanması gerektiğini deneyle gösterdi: `clone()` yerine `return this` yazılınca `setMemento` sonrasında "yeni durum" yazıldı. Hoca nedenini tam anlayamadığını, ama çalışan çözümü bulduğunu söyledi.

  Ek bilgi (derste anlatılmadı): Klonlama gerekliliğinin nedeni büyük olasılıkla takma ad (aliasing) sorunudur. `clone() this` döndürürse Memento ile Originator aynı `State` nesnesini paylaşır. Sonraki `modify("yeni durum")` çağrısı bu ortak nesneyi değiştirdiği için Memento'daki "eski" durum da değişmiş olur. Kopya verildiğinde Memento kendi bağımsız nesnesini tutar.

- Observer kalıbında soyut Subject, gözleyici listesini ve `Attach/Detach/Notify` ortak işlerini tutar; `Notify` bütün gözleyicilerin `Update` metodunu çağırır. Önceki Iterator örneğindeki gibi bir iterator ile listede dolaşmak yeterlidir. Gözleyicinin bilgiyi nasıl alacağı pull ya da push modele bağlıdır. Pull modelde gözlenen yalnız soyut Observer üzerinden

`Update` çağırır; somut gözleyicinin türünü bilmesi gerekmez, gözleyici de yalnız ilgilendiği alanları sorgular. Ama gözleyicinin ilgilenmediği bir alan değişse bile bildirim gider; gözleyici sorgulayıp hiçbir şeyin değişmediğini görebilir. Bu yüzden boşuna yapılan işin masraflı olmaması gerekir. Push modelde bütün durum gönderildiği için daha çok bayt gider ve haberleşme uzar; bir değişiklik maskesi gözleyicinin işini kolaylaştırır.

- Tetikleme sorununda subject tetiklerse, durumu değiştiren her metodun sonunda `Notify` çağrılmak zorundadır. Durum çok sık değişiyorsa ya da gözleyiciler anlık değişikliklerle ilgilenmiyorsa bu gereksiz trafik doğurur. Değişiklikleri biriktirip toplu göndermek mümkündür, ama `SetState/Notify` karmaşıklaşır. Hoca, haberleşme masrafı için optimizasyon yapılacaksa en uygun yerin yine gözlenen nesne olduğunu, kazanç önemliyse bu karmaşıklığın göze alınacağını söyledi.
- Müşteri ilişkileri örneğinde arayüzün adı `ICObserver`'dır: I arayüzü, C müşteriyi (Customer) gösterir. Hoca arayüzü yalnız müşteri gözleyicisi diye kısıtlamanın sorun olmadığını söyledi. Örneğin bir şube (location) türü subject eklenirse onun için ayrı bir gözleyici arayüzü tanımlanır, `AddrVerification` gibi bir sınıf iki arayüzü birden gerçekleyebilir. Ama birden fazla subject türü olacaksa `attach/detach` ve gözleyici listesinin her sınıfta tekrarlanmaması (duplicate code) için soyut Subject sınıfı mutlaka eklenmelidir. Örnekte gözleyici listesi bütün müşteriler için ortak olduğundan statiktir; bu yüzden `attach` ve `detach` de statiktir. Kalıp bunu gerektirmez.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.